This assignment can be completed on **one Ubuntu EC2 instance** running:

- Flask Backend → Port **5000**
- Express Frontend → Port **3000**
- Jenkins → Port **8080**
- Nginx (optional reverse proxy) → Port **80**

==

Architecture:

```
            GitHub
              |
        GitHub Webhook
              |
              v
         Jenkins (8080)
              |
      -----------------------
      |                   |
      v                   v
 Flask Backend       Express Frontend
   (5000)               (3000)
      |                   |
      ------- EC2 ----------
           Ubuntu
```

PART 1 – Deploy Flask and Express on EC2

Step 1: Launch EC2

Launch an Ubuntu Server 22.04 LTS EC2 instance.

Select t2.micro as the instance type (Free Tier eligible).

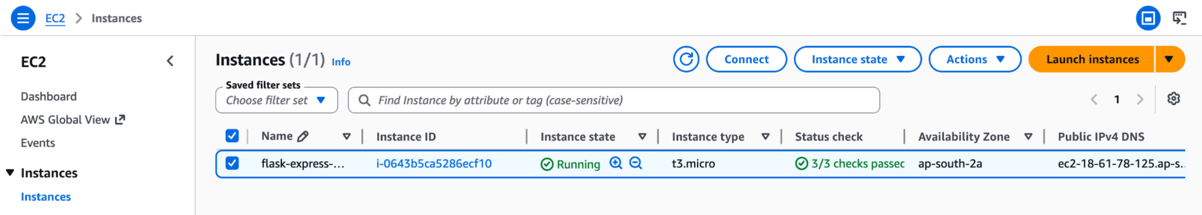
Configure 20 GB of storage.

Create or select an existing key pair for SSH access.

Configure the Security Group with the following inbound rules:

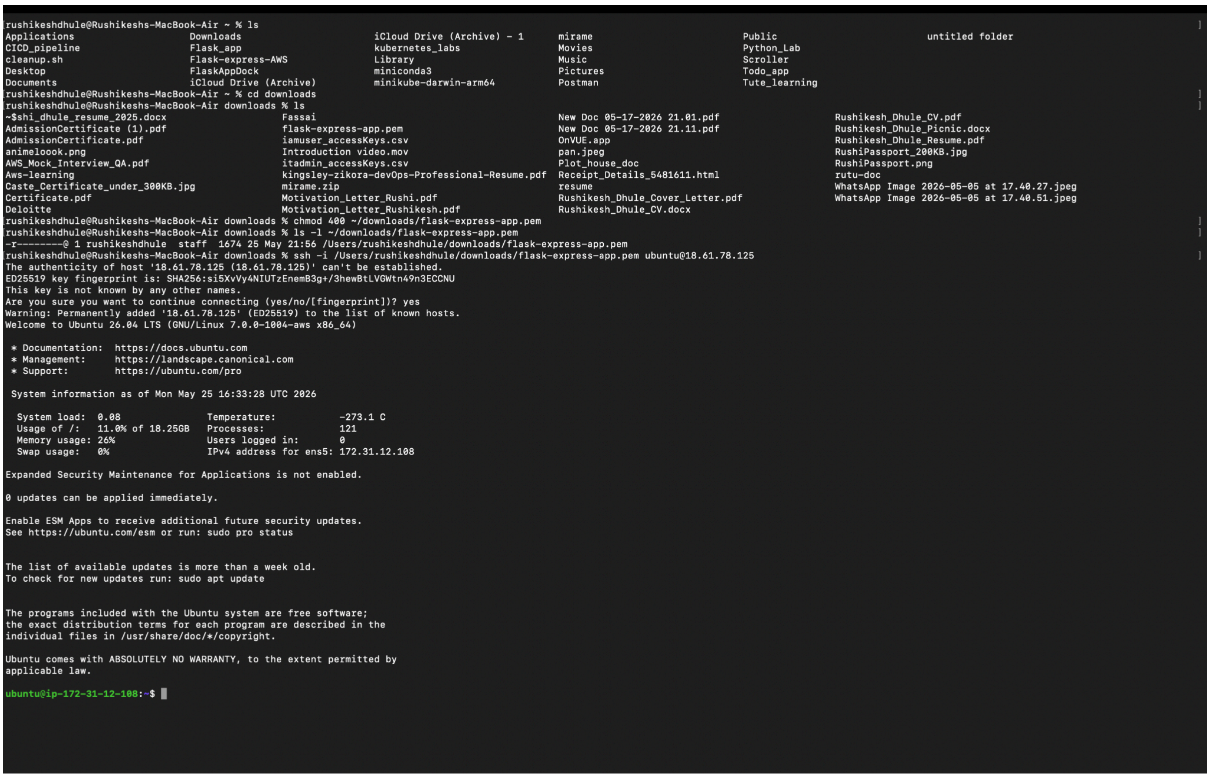
- Port 22 – SSH access
- Port 80 – Website (HTTP)
- Port 3000 – Express Frontend
- Port 5000 – Flask Backend
- Port 8080 – Jenkins Dashboard

Launch the instance and note the public IP address.



Step 2: Connect EC2

ssh -i mykey.pem ubuntu@PUBLIC_IP



Step 3: Update Server

sudo apt update
sudo apt upgrade -y

Step 4: Install Git

sudo apt install git -y

Step 5: Install Python

sudo apt install python3 python3-pip python3-venv -y

Step 6: Install NodeJS

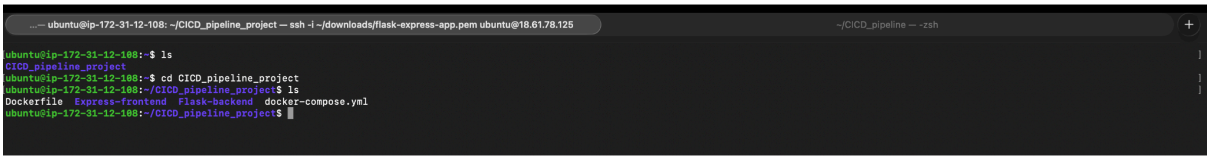
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install nodejs -y

Step 7: Install PM2

sudo npm install -g pm2

Step 8: Clone Repositories

Git clone https://github.com/21Rushi/CICD_pipeline_project.git

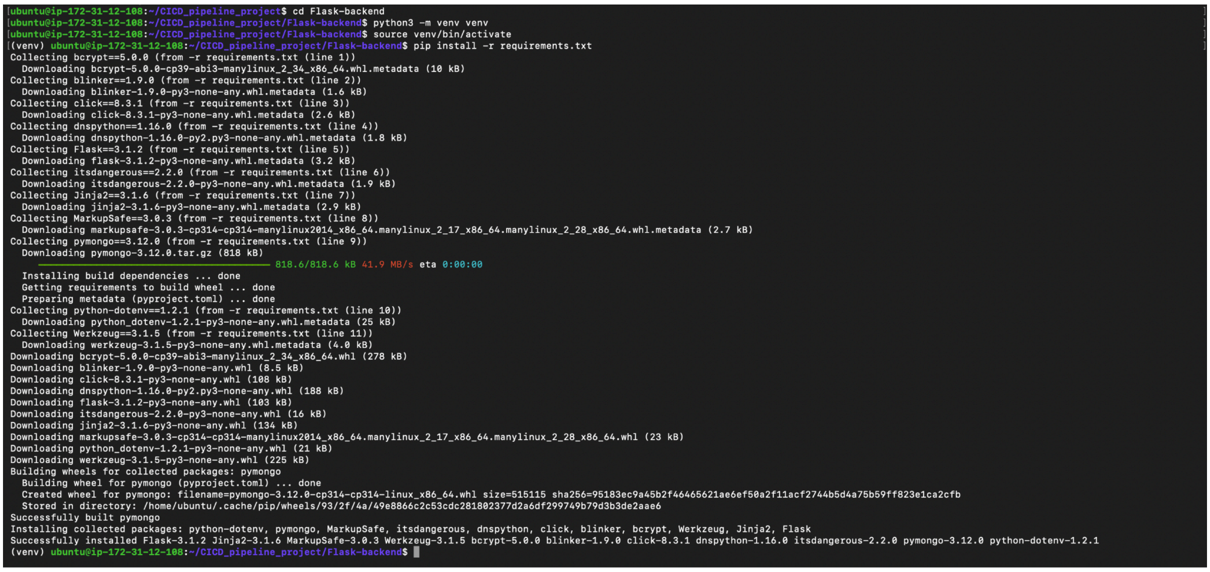


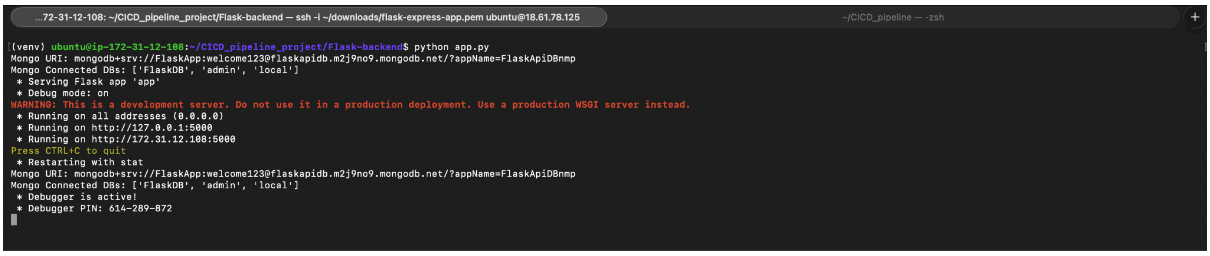
Step 9: Deploy Flask Backend

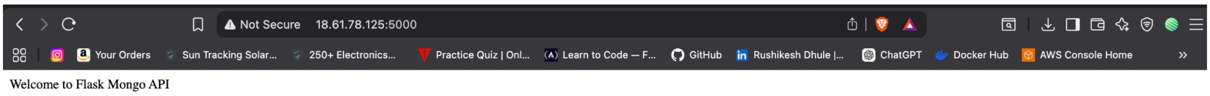
cd flask-backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

Test→ python [app.py](app.py)

http://18.61.78.125:5000/



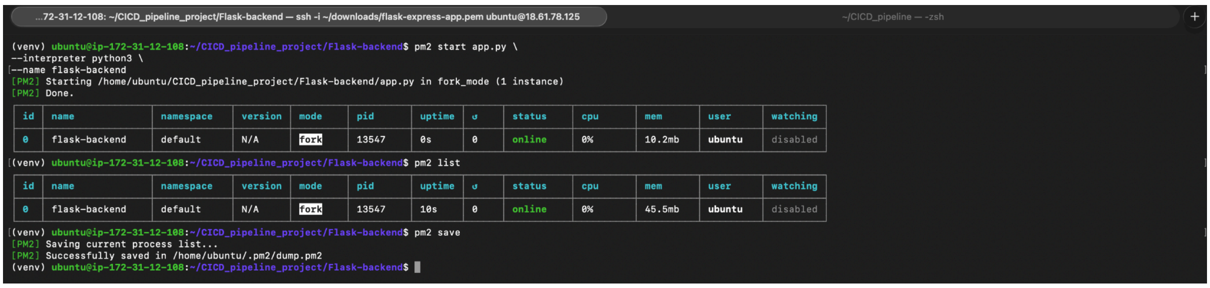




Step 10: Run Flask Using PM2

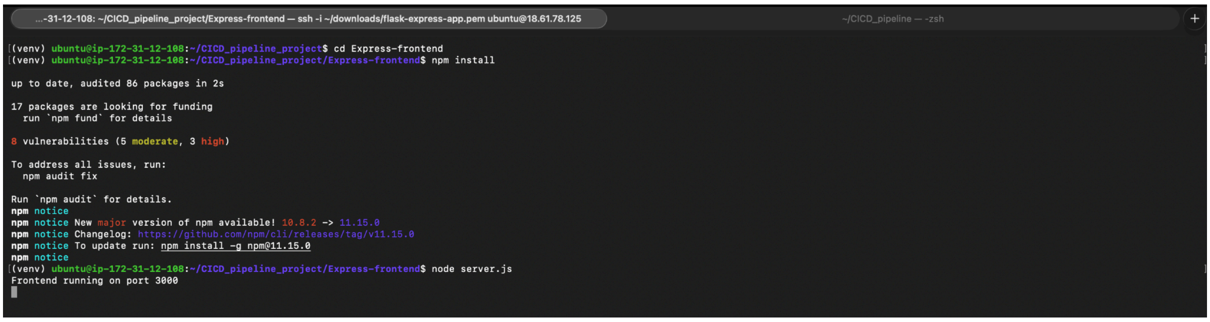
pm2 start app.py \
--interpreter python3 \
--name flask-backend

Check → pm2 list
Save → pm2 save

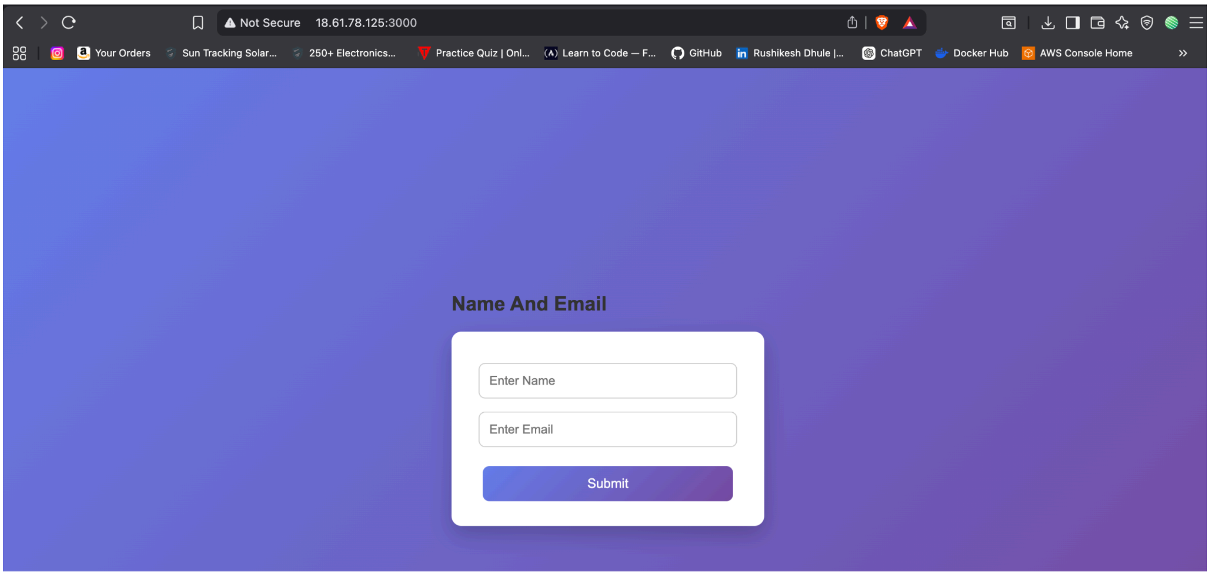


Step 11: Deploy Express Frontend

cd ~/projects/express-frontend
npm install
Start → node server.js



Check → http://18.61.78.125:3000/

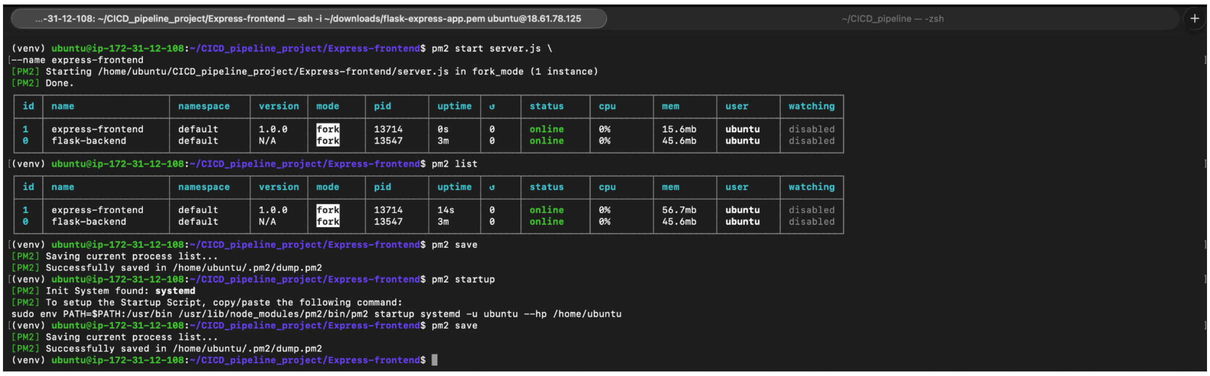


Step 12: Run Express Using PM2

pm2 start server.js \
--name express-frontend

Check→ pm2 list
Save → pm2 save

Enable startup → pm2 startup
Save → pm2 save



```
...-31-12-108: ~/CICD_pipeline_project/Express-frontend — ssh -i ~/downloads/flask-express-app.pem ubuntu@18.61.78.125    ~/CICD_pipeline — -zsh

(venv) ubuntu@ip-172-31-12-108:~/CICD_pipeline_project/Express-frontend$ pm2 start server.js \
--name express-frontend
[PM2] Starting /home/ubuntu/CICD_pipeline_project/Express-frontend/server.js in fork_mode (1 instance)
[PM2] Done.
│ id │ name             │ namespace │ version │ mode │ pid   │ uptime │ ↺ │ status │ cpu │ mem    │ user   │ watching │
│ 1  │ express-frontend │ default   │ 1.0.0   │ fork │ 13714 │ 0s     │ 0 │ online │ 0%  │ 15.6mb │ ubuntu │ disabled │
│ 0  │ flask-backend    │ default   │ N/A     │ fork │ 13547 │ 3m     │ 0 │ online │ 0%  │ 45.6mb │ ubuntu │ disabled │
(venv) ubuntu@ip-172-31-12-108:~/CICD_pipeline_project/Express-frontend$ pm2 list
│ id │ name             │ namespace │ version │ mode │ pid   │ uptime │ ↺ │ status │ cpu │ mem    │ user   │ watching │
│ 1  │ express-frontend │ default   │ 1.0.0   │ fork │ 13714 │ 14s    │ 0 │ online │ 0%  │ 56.7mb │ ubuntu │ disabled │
│ 0  │ flask-backend    │ default   │ N/A     │ fork │ 13547 │ 3m     │ 0 │ online │ 0%  │ 45.6mb │ ubuntu │ disabled │
(venv) ubuntu@ip-172-31-12-108:~/CICD_pipeline_project/Express-frontend$ pm2 save
[PM2] Saving current process list...
[PM2] Successfully saved in /home/ubuntu/.pm2/dump.pm2
(venv) ubuntu@ip-172-31-12-108:~/CICD_pipeline_project/Express-frontend$ pm2 startup
[PM2] Init System found: systemd
[PM2] To setup the Startup Script, copy/paste the following command:
sudo env PATH=$PATH:/usr/bin /usr/lib/node_modules/pm2/bin/pm2 startup systemd -u ubuntu --hp /home/ubuntu
(venv) ubuntu@ip-172-31-12-108:~/CICD_pipeline_project/Express-frontend$ pm2 save
[PM2] Saving current process list...
[PM2] Successfully saved in /home/ubuntu/.pm2/dump.pm2
(venv) ubuntu@ip-172-31-12-108:~/CICD_pipeline_project/Express-frontend$
```

PART 2 – Install Jenkins

Step 13: Install Java

sudo apt update
sudo apt install fontconfig openjdk-21-jre
java -version

```
sudo apt update
sudo apt install fontconfig openjdk-21-jre
java -version
```

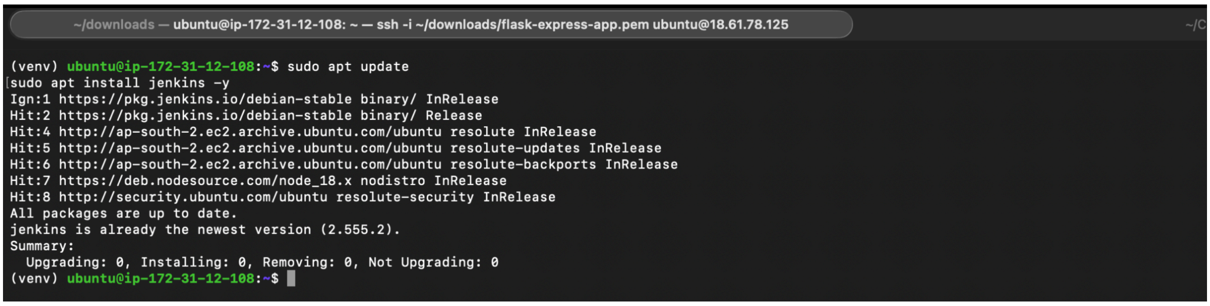
Step 14: Install Jenkins

sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update
sudo apt install jenkins

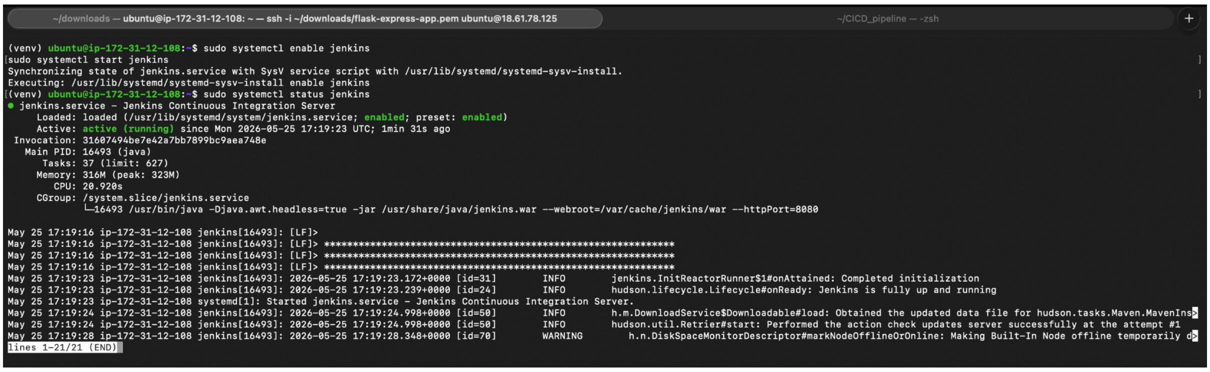
```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update
sudo apt install jenkins
```

Start →

sudo systemctl enable jenkins
sudo systemctl start jenkins



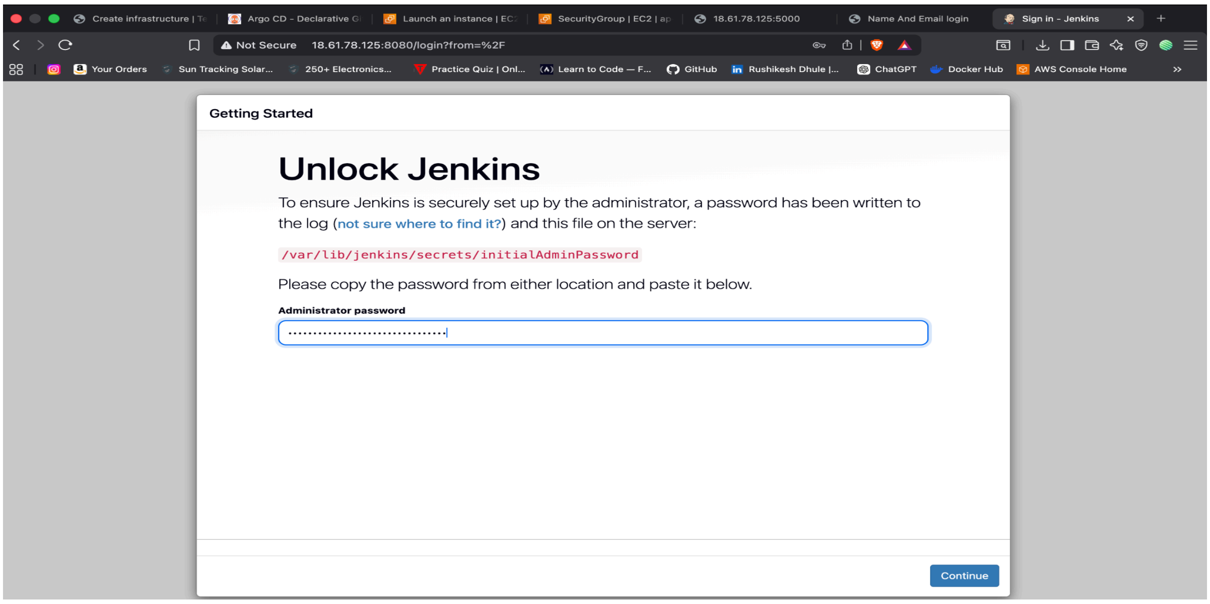
Verify status : sudo systemctl status jenkins



Step 15: Get Jenkins Password

sudo cat   /var/lib/jenkins/secrets/initialAdminPassword

Check → http://18.61.78.125:8080/



Step 16: Install Jenkins Plugins

Dashboard → Manage Jenkins → Plugins

Install:

- Git Plugin
- Pipeline Plugin
- NodeJS Plugin
- SSH Agent Plugin

Restart Jenkins.

Step 17: Configure Tools

Manage Jenkins → Tools

NodeJS:

NodeJS18

Install automatically.

Python already exists on server.

PART 3 – CI/CD Pipeline :

Jenkins Script →

```
pipeline {
    agent any

    stages {

        stage('Clone Repository') {
            steps {
                git branch: 'main',
                url: 'https://github.com/21Rushi/CICD_pipeline_project.git'
            }
        }

        stage('Install Backend Dependencies') {
```

```
    steps {
        dir('Flask-backend') {
            sh '''
            pip3 install --break-system-packages -r requirements.txt
            '''
        }
    }
}

        stage('Install Frontend Dependencies') {
            steps {
                dir('Express-frontend') {
                    sh '''
                    npm install
                    '''
                }
            }
        }

        stage('Deploy Backend') {
            steps {
                dir('Flask-backend') {
                    sh '''
                    pm2 restart flask-backend || \
                    pm2 start app.py \
                    --interpreter python3 \
                    --name flask-backend
                    '''
                }
            }
        }

        stage('Deploy Frontend') {
            steps {
                dir('Express-frontend') {
                    sh '''
                    pm2 restart express-frontend || \
                    pm2 start npm \
                    --name express-frontend \
                    -- start
                    '''
                }
            }
        }
    }

    post {
        success {
```

```
            echo 'Deployment Successful'
        }

        failure {
            echo 'Deployment Failed'
        }
    }
}
```

Step 18: Configure GitHub Webhook

GitHub Repository →

Settings →

Webhooks →

Add webhook

Payload URL:

http://PUBLIC_IP:8080/github-webhook/

Content type:

application/json

Events:

Just the push event

Save.

Step 19: Enable GitHub Trigger in Jenkins

Pipeline →

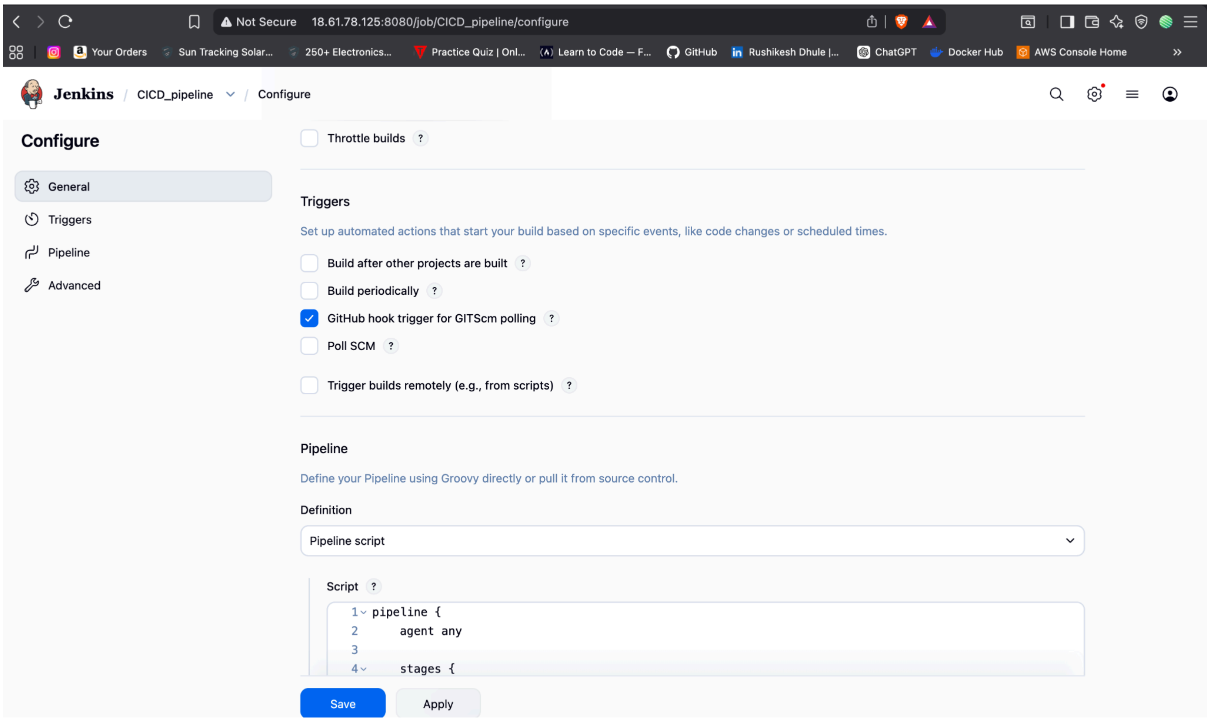
Configure →

Build Triggers →

Enable:

GitHub hook trigger for GITScm polling

Save.



Step 20: Test CI/CD

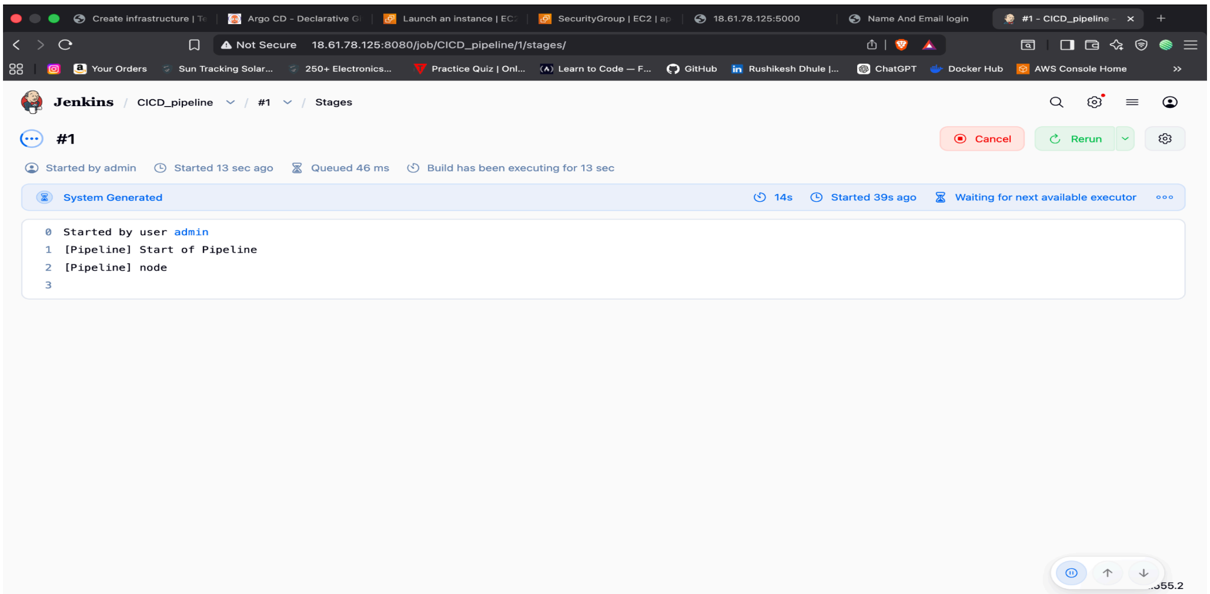
Push code:

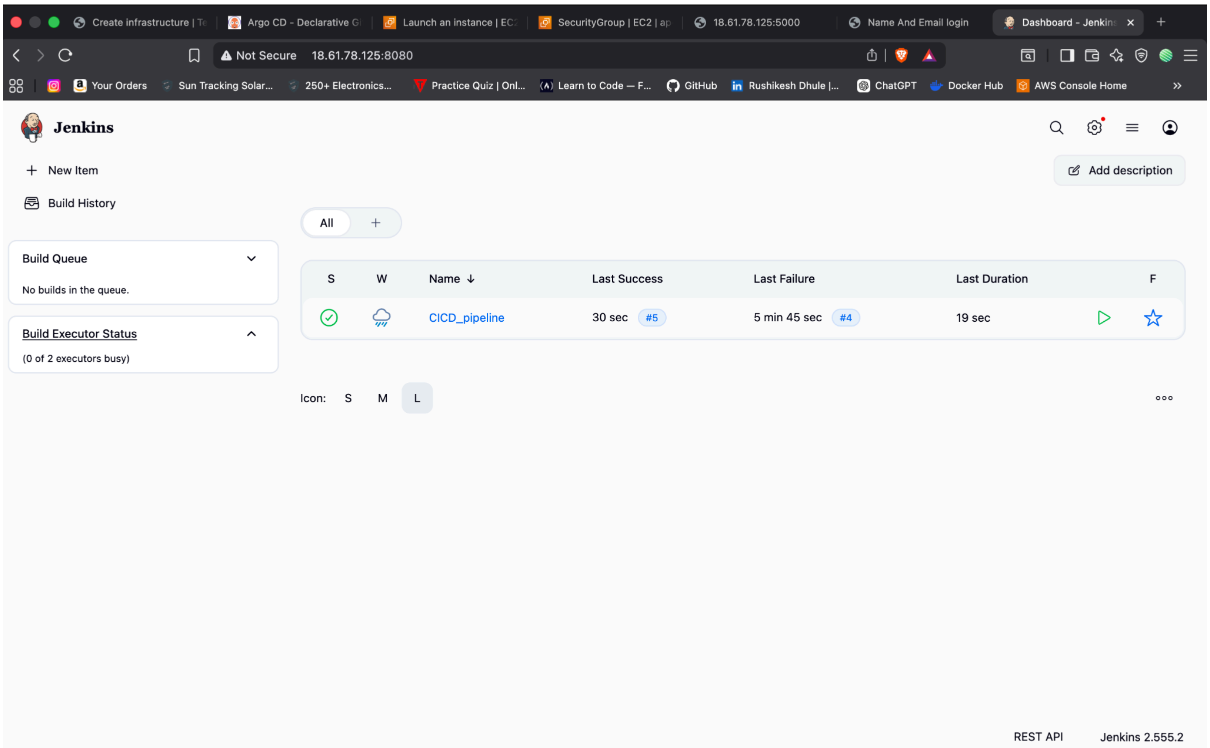
```
git add .
git commit -m "testing ci"
git push origin main
```

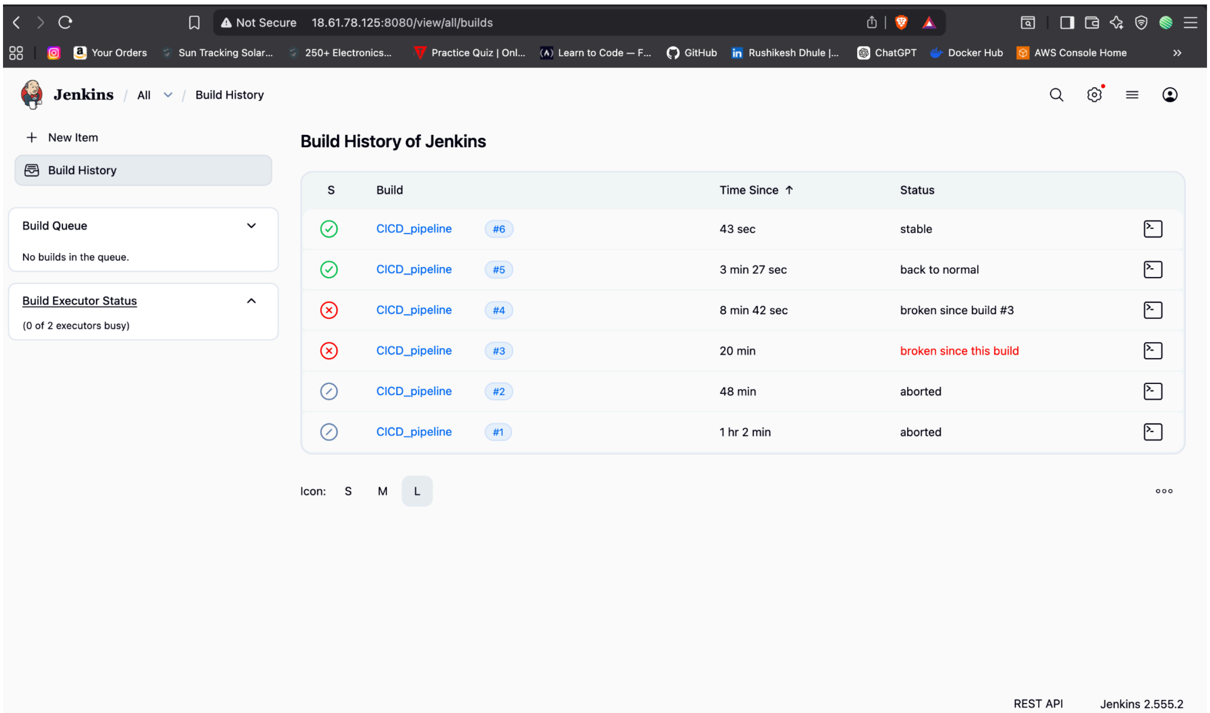
Flow →

Git Push
↓
GitHub Webhook
↓
Jenkins Trigger
↓
Clone Latest Code
↓
Install Dependencies
↓
Restart Application
↓
Deployment Complete

Reviewed application logs, performed troubleshooting, identified and resolved build issues, and ensured the CI/CD pipeline completed successfully.







Github link : https://github.com/21Rushi/CICD_pipeline_project

Google link :
https://docs.google.com/document/d/1WzWsH_RI1tVyfW1xaNG5Oo58hhMQgLyEbMeF3Dk6JaM/edit?tab=t.wrz9m7vyus06